

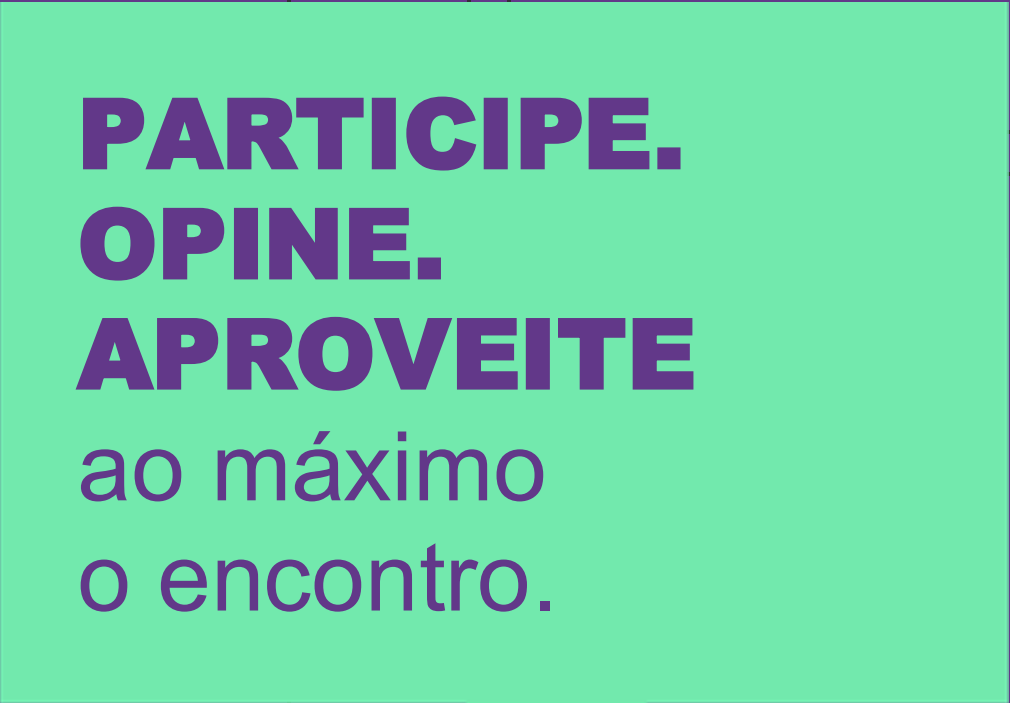
Programação em Python

Desenvolver Aplicações Back- End para Web com Python

Bem-vindos à Aula:
Desenvolver Aplicações
Back-End para web com
python

O que faremos hoje:

- Entender o que é um banco de dados
- Entender tabelas e registros
- Compreender relacionamentos
- Criar modelos (Models)
- Gerar migrations
- Aplicar migrations
- Visualizar tabelas no banco
- Entender a relação entre Models e Banco de Dados



**PARTICIPE.
OPINE.
APROVEITE**
ao máximo
o encontro.

Estamos em um ambiente
heterogêneo e diverso,
onde sempre há respeito.

Então, lembre-se de que:

**NÃO EXISTE
RESPOSTA ERRADA,
PERGUNTA BOBA**
ou que não valha a pena.

Para não termos interrupções e em respeito aos colegas,
pedimos que não faça uso de celular durante a aula e que
o coloque no modo silencioso.



O que é um banco de dados?



Reflexão Inicial

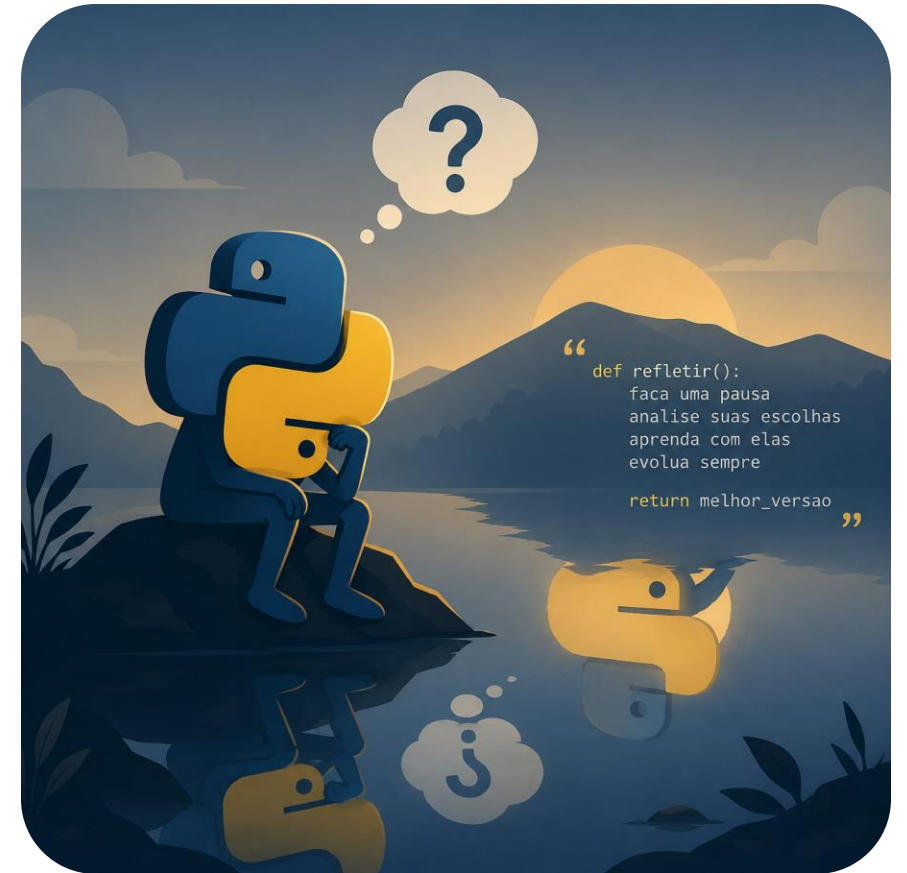
Imagine uma escola.

A escola precisa armazenar:

- alunos
- professores
- turmas
- notas

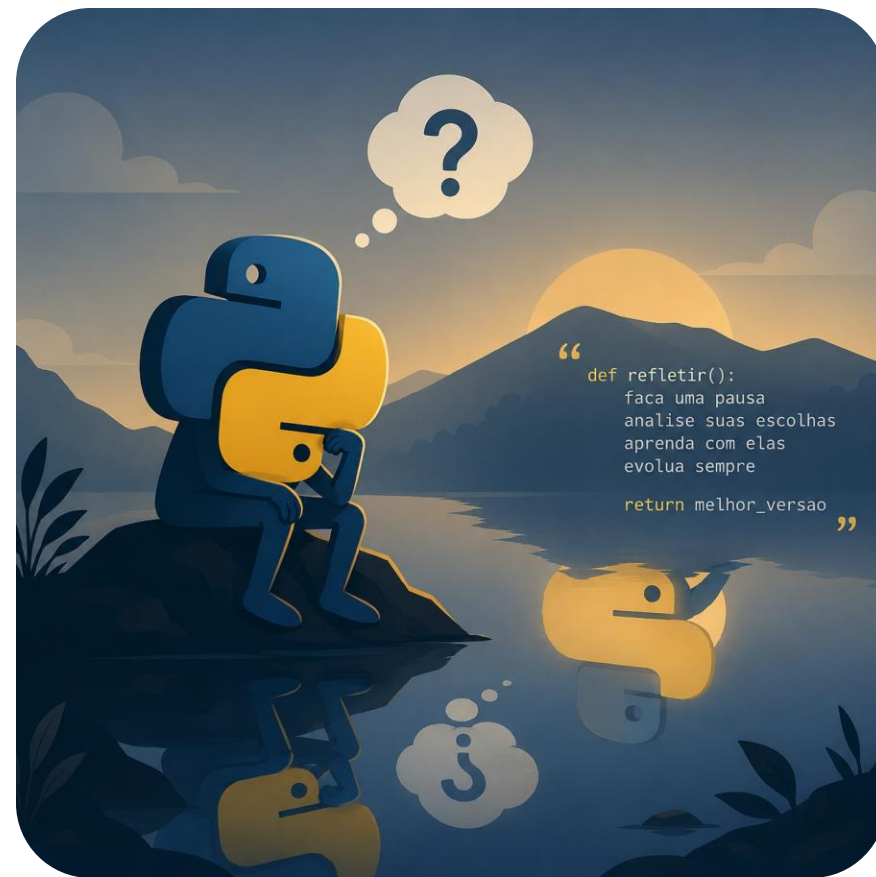
Se tudo fosse anotado em papéis soltos, seria difícil encontrar informações.

Por isso utilizamos um banco de dados.



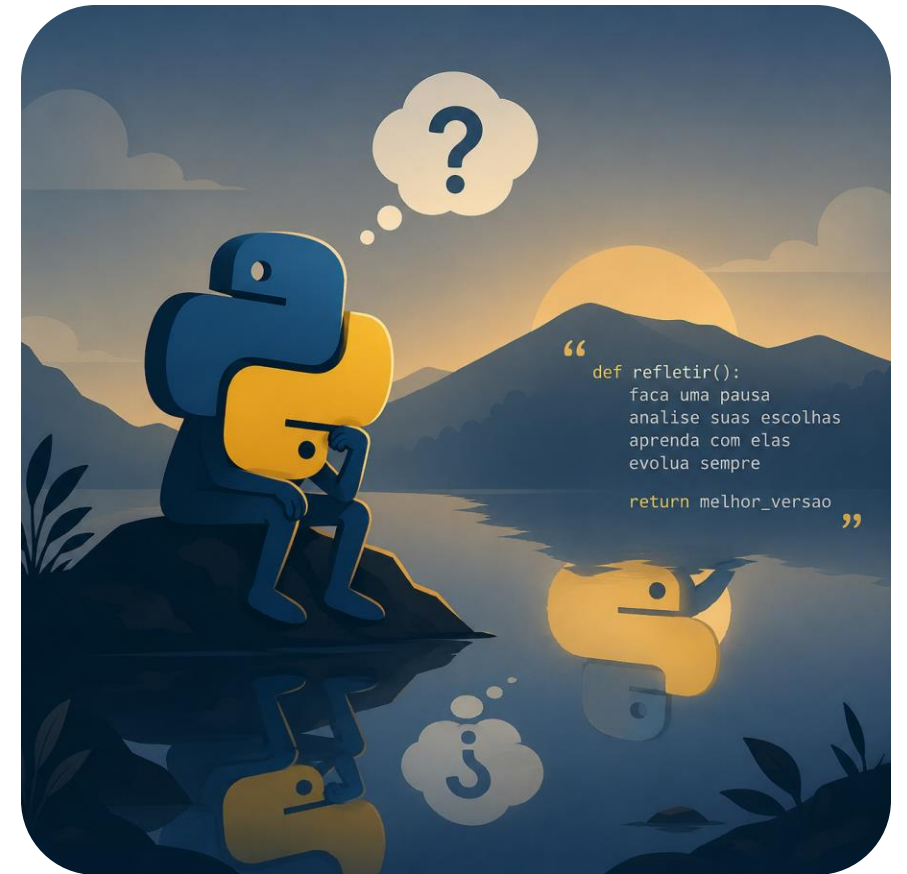
O que é um Banco de Dados?

Banco de dados é um local onde armazenamos informações de forma organizada.



Vida Real

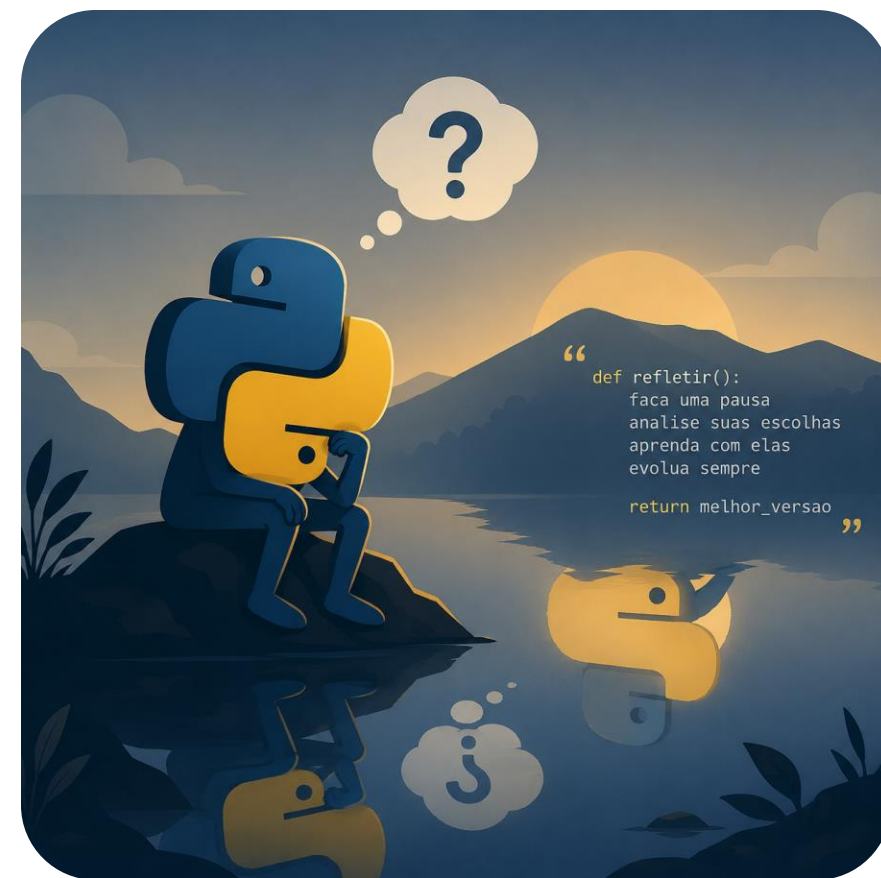
Local	Banco de Dados
Agenda telefônica	Contatos
Biblioteca	Livros
Banco	Contas
Hospital	Pacientes
Escola	Alunos



Exemplo

Tabela de Clientes

id	nome	idade
1	João	18
2	Maria	20
3	Pedro	25

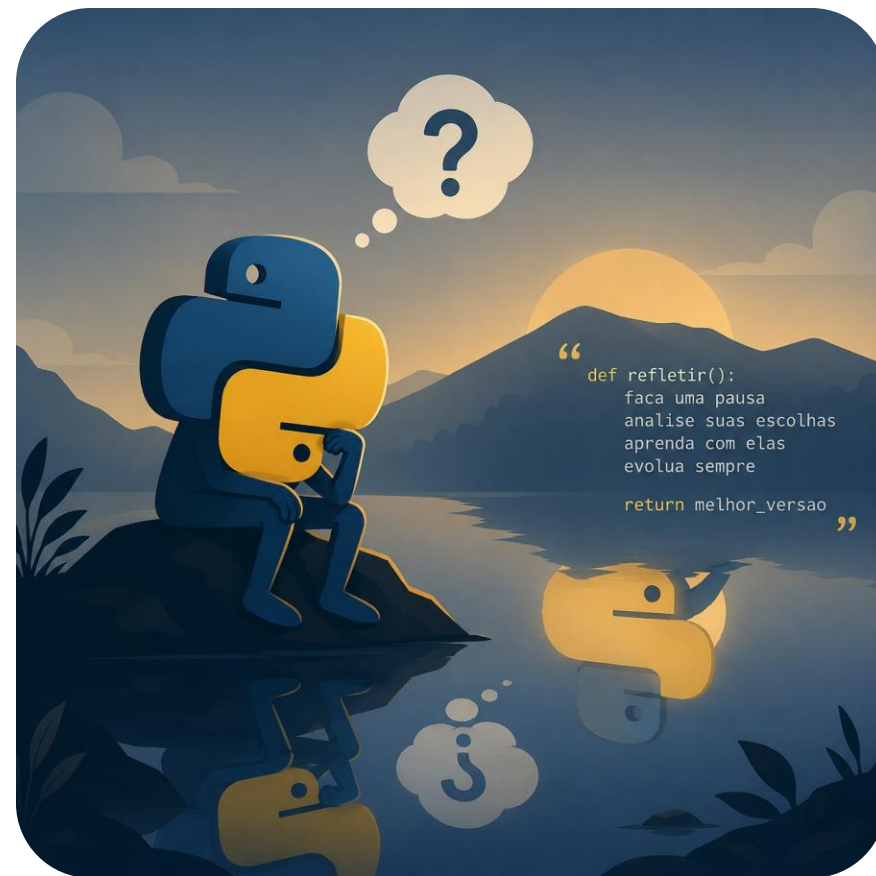


Reflexão

Uma tabela funciona como uma planilha.

Cada linha representa uma pessoa.

Cada coluna representa uma característica.



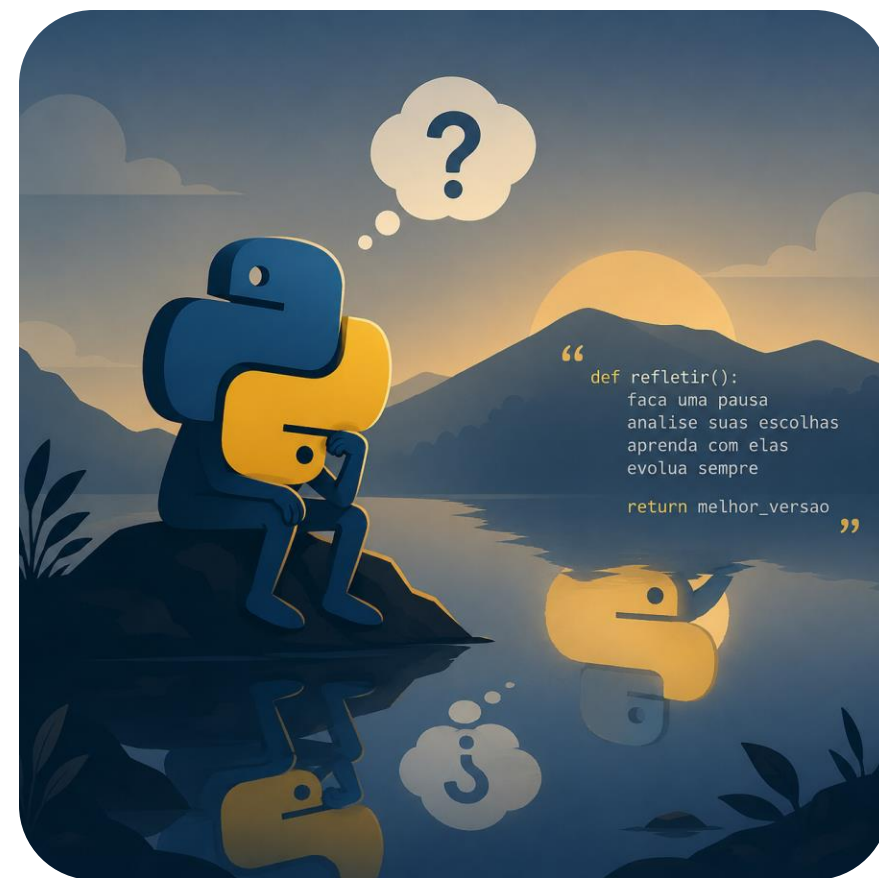
Conceitos Importantes

Registro

Uma linha da tabela.

Exemplo:

1 | João | 18

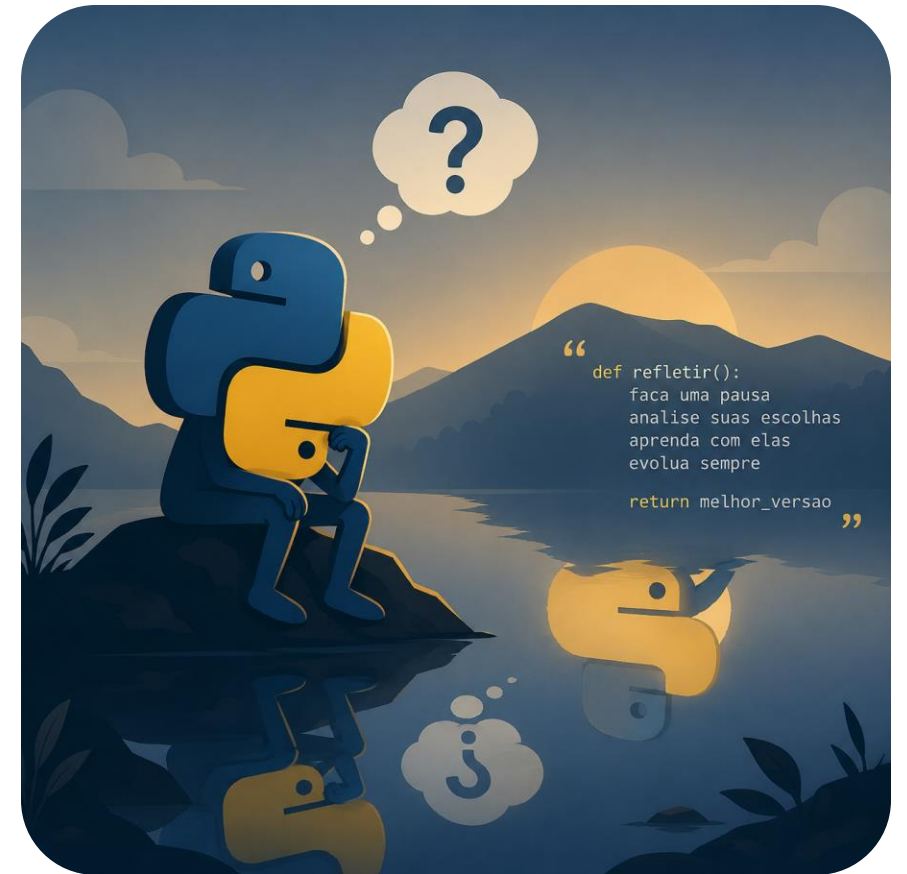


Campo

Uma coluna da tabela.

Exemplo:

nome
idade



A cluster of five purple squares of varying sizes arranged in a loose, abstract pattern around the word "Desafio".

Desafio

Desafio

Imagine um sistema de biblioteca.

Quais informações seriam necessárias para cadastrar um livro?



- título
- autor
- ano
- editora



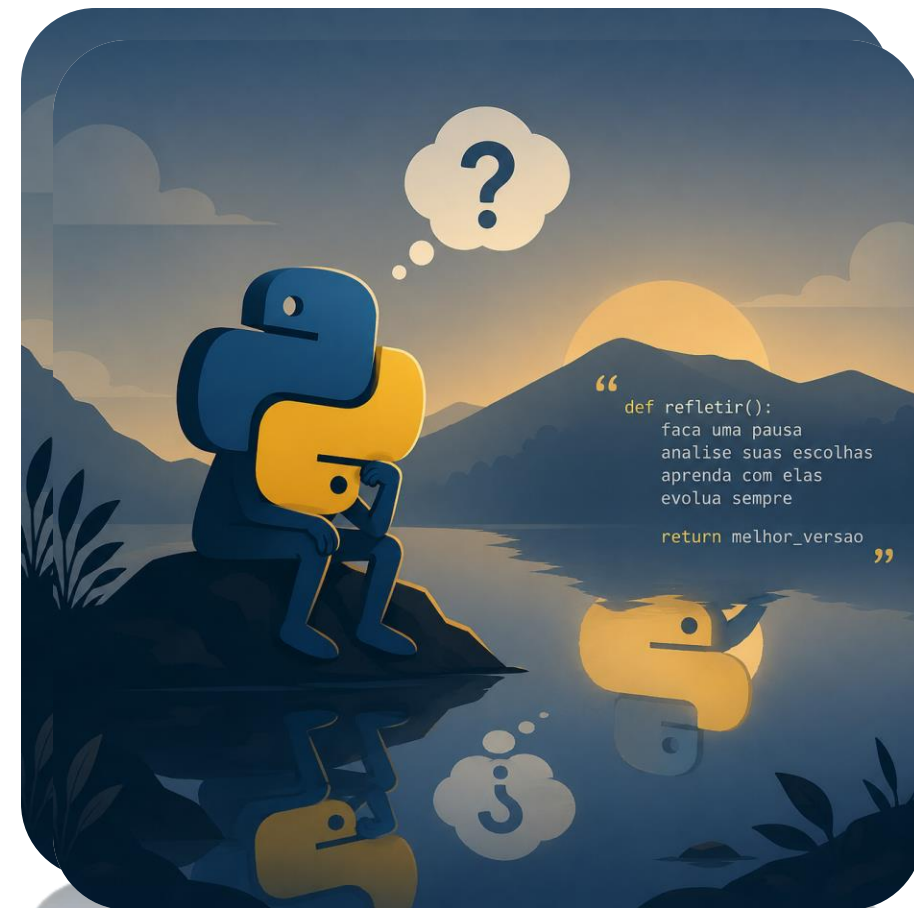


O que são relacionamentos?



O que é um relacionamento?

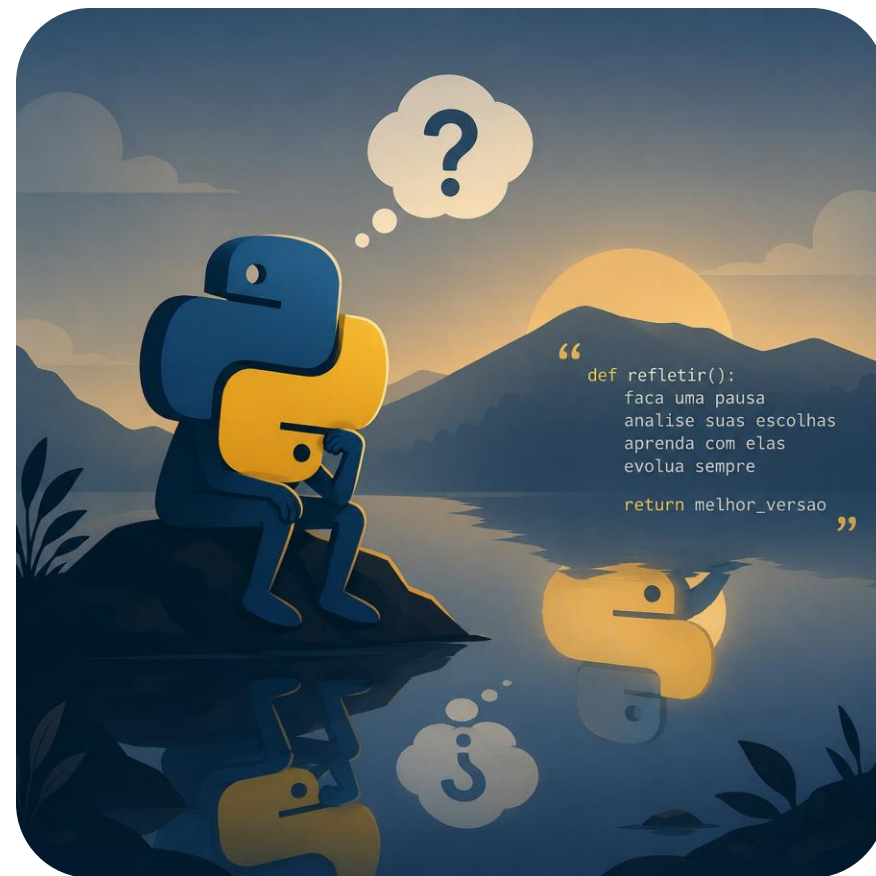
Relacionamento é quando uma informação depende de outra.



Vida Real

Imagine uma escola.

Um aluno pertence a uma turma.



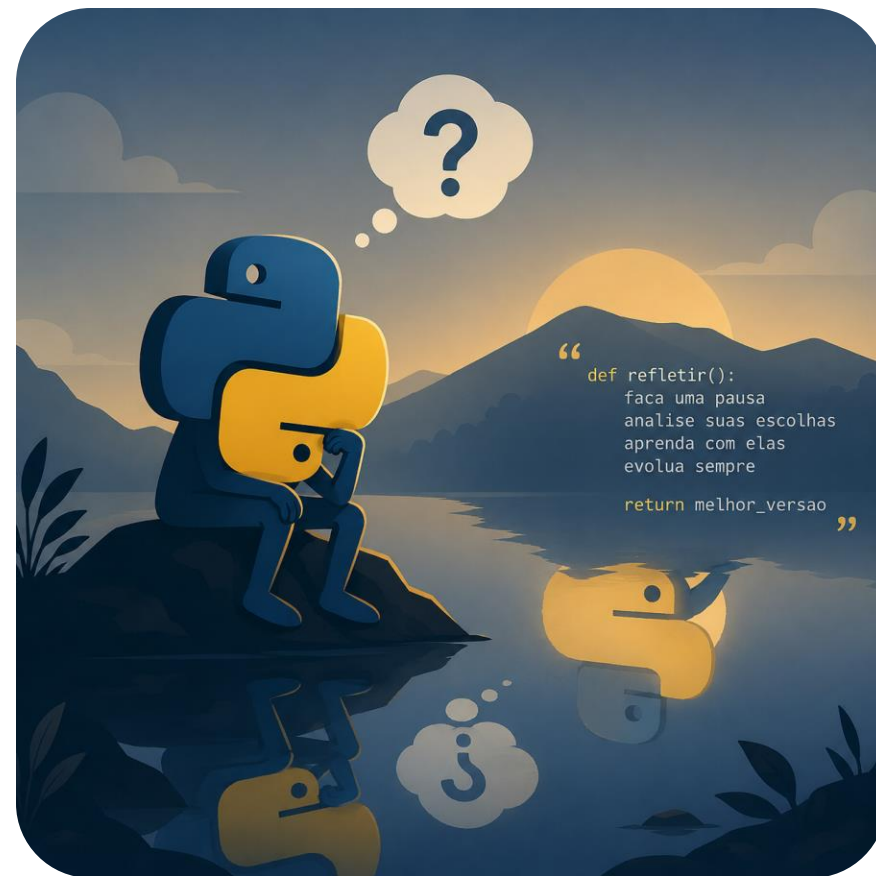
```
“def refletir():  
    faça uma pausa  
    analise suas escolhas  
    aprenda com elas  
    evolua sempre  
    return melhor-versao”
```

Tabela Turma

id	nome
1	ADS A
2	ADS B

Tabela Aluno

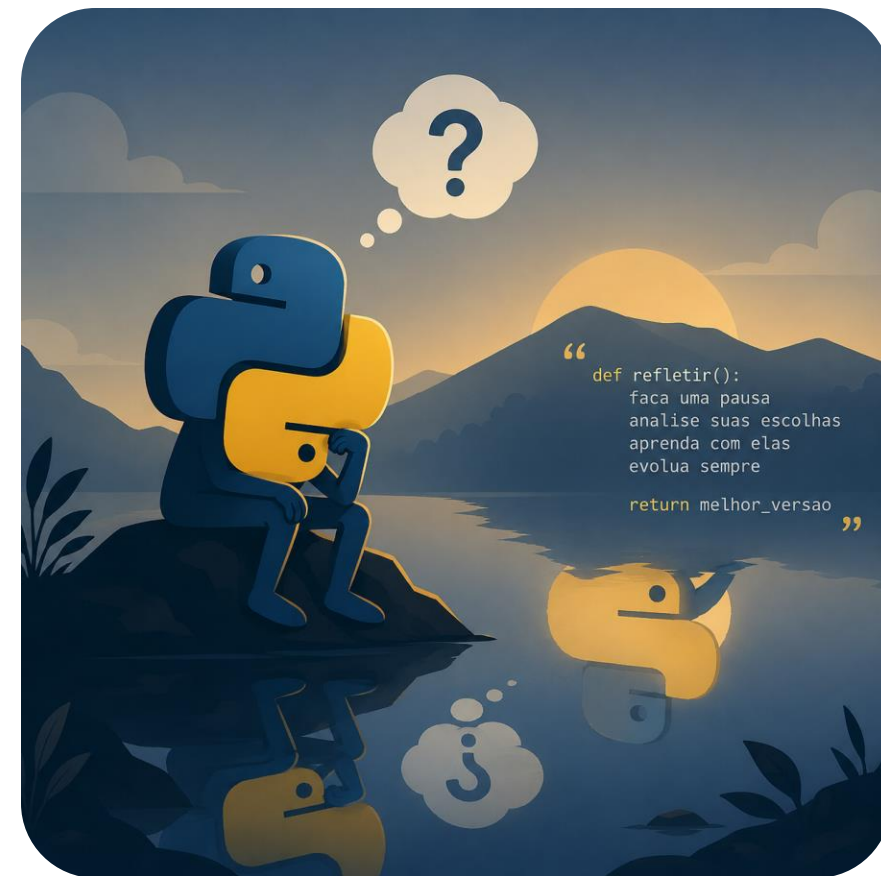
id	nome	turma
1	João	1
2	Maria	2



Reflexão

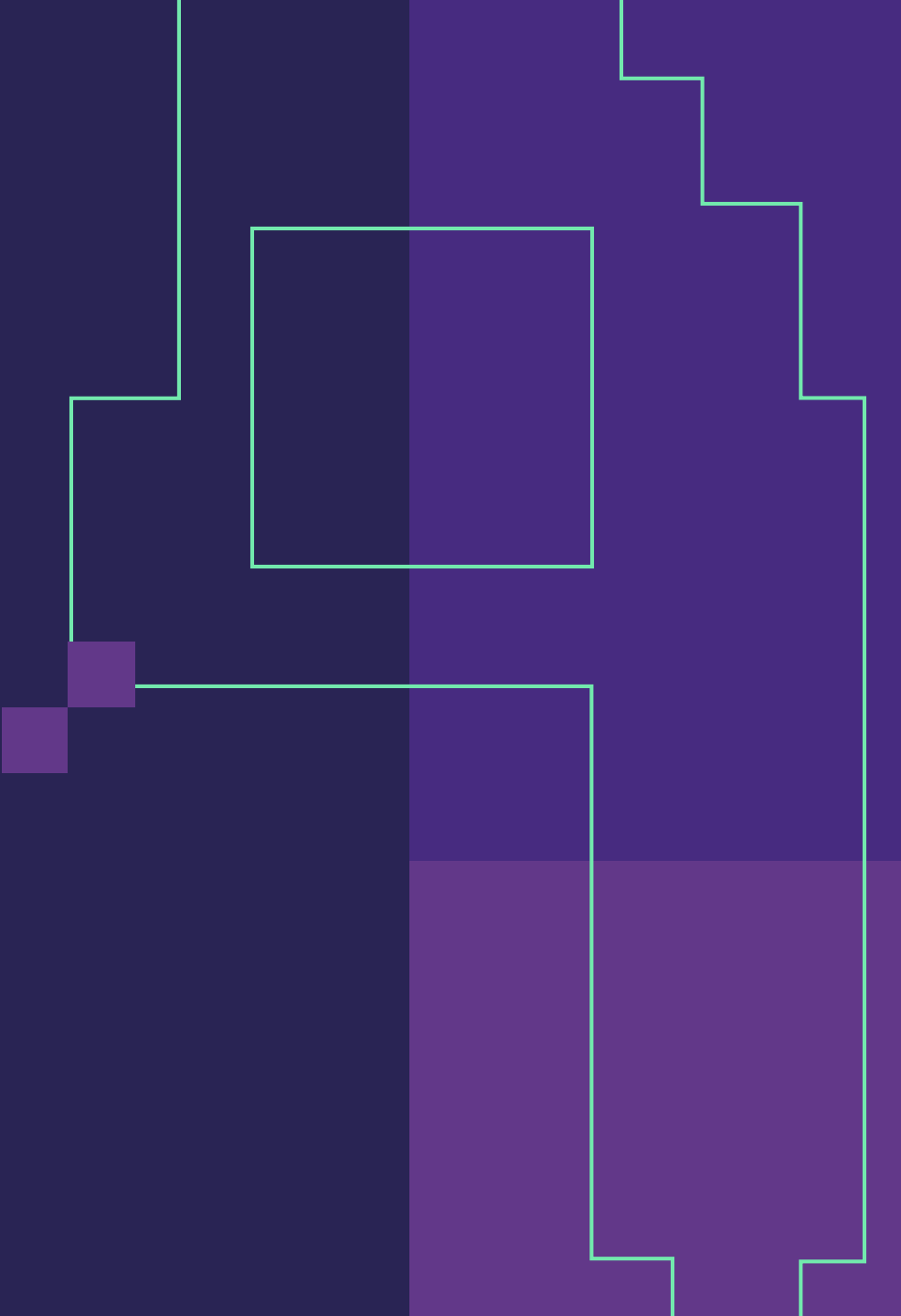
Um aluno está ligado a uma turma.

Isso é um relacionamento.





Tipos de relacionamentos



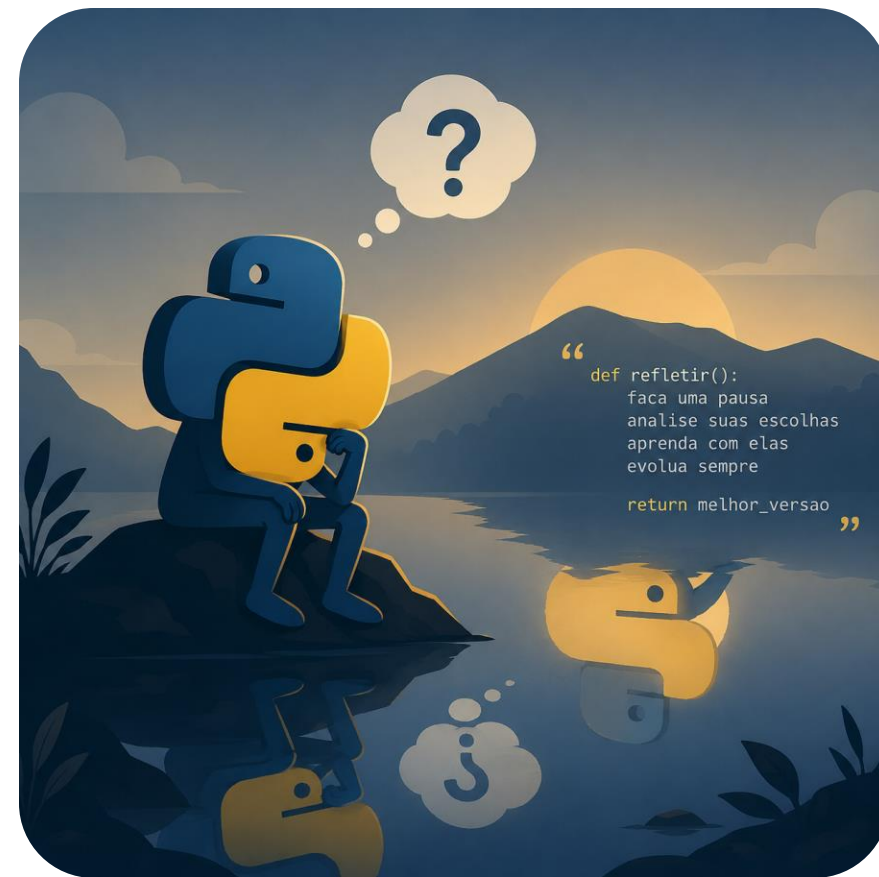
Um para Um (OneToOne)

Vida Real

Pessoa → CPF

Uma pessoa possui apenas um CPF.

Um CPF pertence a apenas uma pessoa.



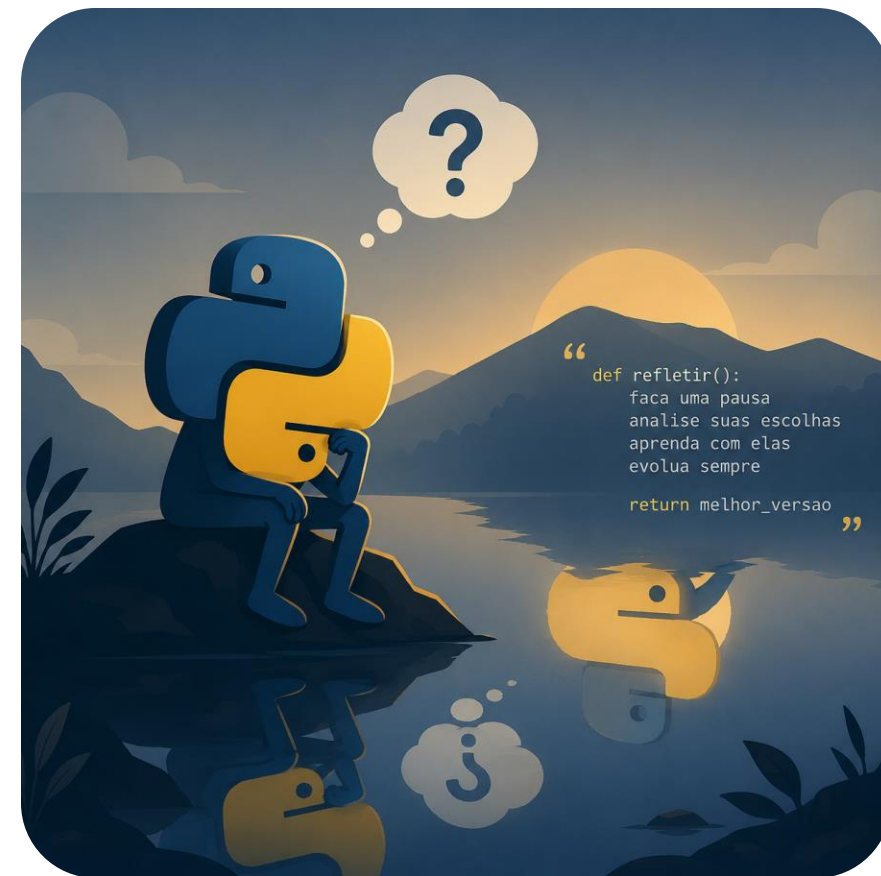
Um para Muitos (ForeignKey)

Vida Real

Professor → Alunos

Um professor possui vários alunos.

Um aluno possui apenas um professor responsável.



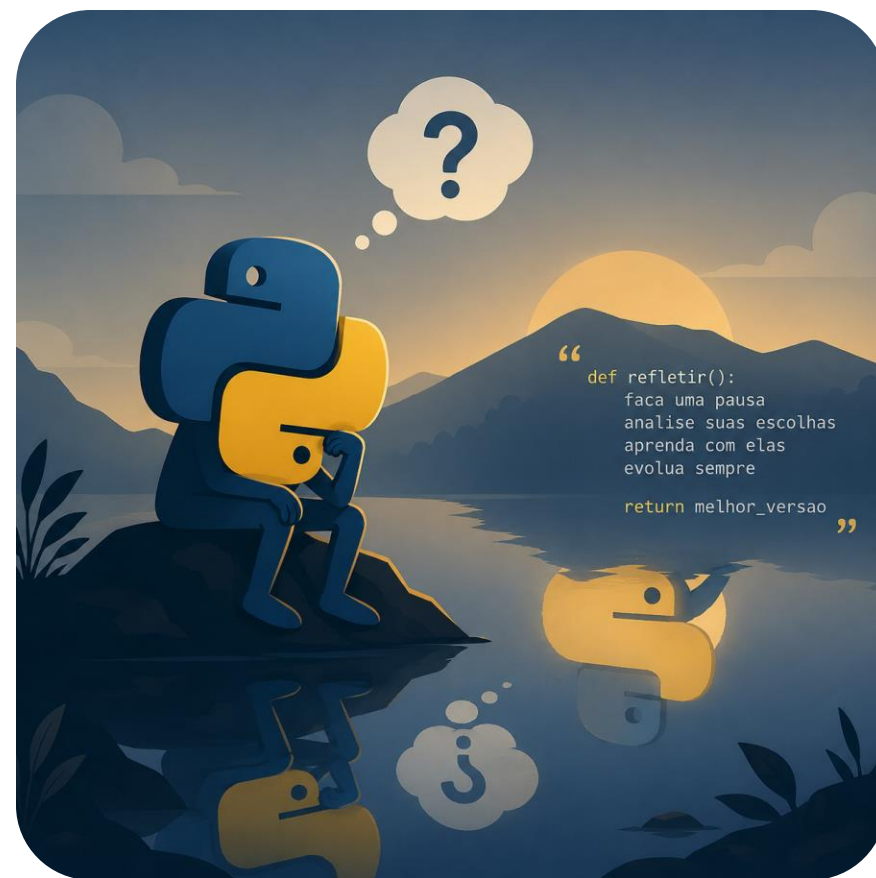
Muitos para Muitos (ManyToMany)

Vida Real

Aluno → Disciplinas

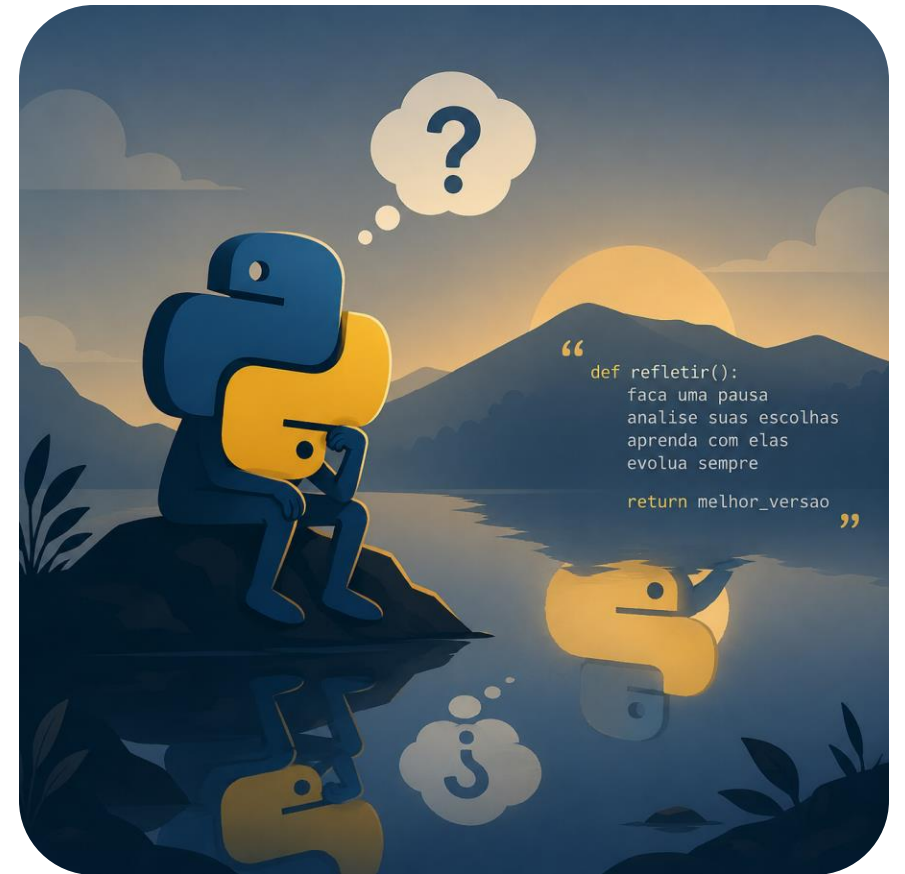
Um aluno cursa várias disciplinas.

Uma disciplina possui vários alunos.



REFLEXÃO

Esses relacionamentos são usados em praticamente todos os sistemas.

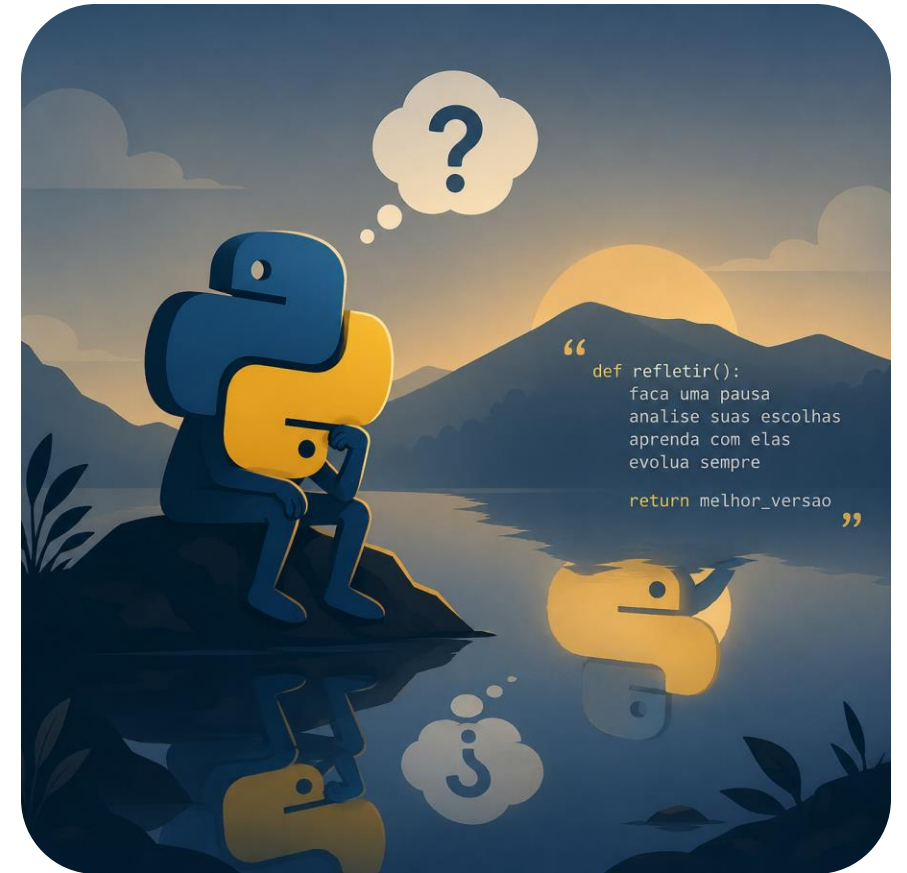




Models no Django

O que é um Model?

Model é a representação de uma tabela do banco de dados.



Django

</> Python

```
class Cliente(models.Model):  
  
    nome = models.CharField(max_length=100)  
  
    idade = models.IntegerField()
```



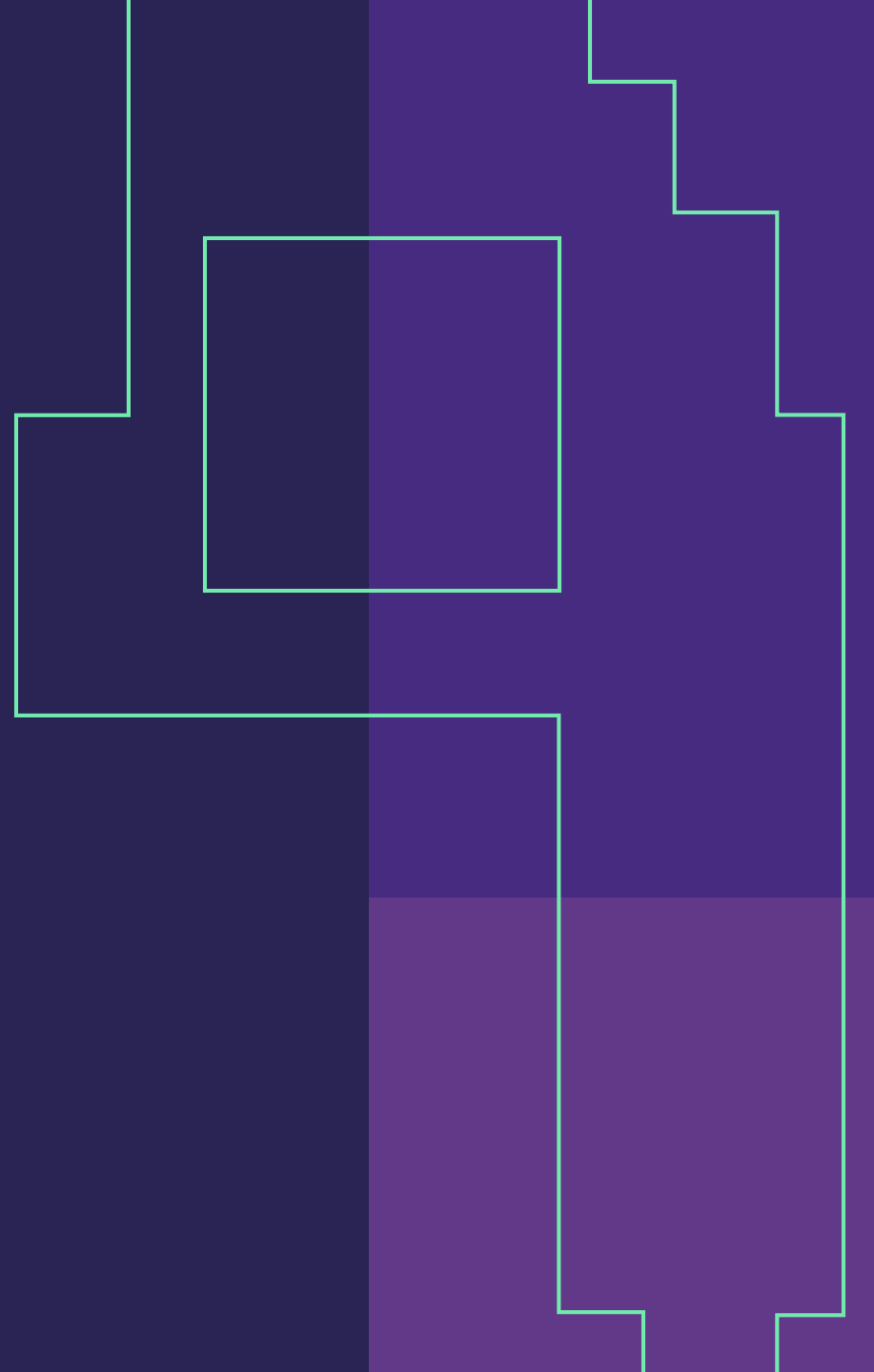
Reflexão

O Django transforma essa classe em uma tabela.



A cluster of five small, solid purple squares arranged in a cross-like pattern around the word "Desafio".

Desafio



Desafio

Criar um Model chamado Livro.

Campos:

- titulo
- autor
- ano



Solução

`</>` Python

```
from django.db import models

class Livro(models.Model):

    titulo = models.CharField(max_length=100)

    autor = models.CharField(max_length=100)

    ano = models.IntegerField()
```



REFLEXÃO

O model funciona como uma planta da tabela.



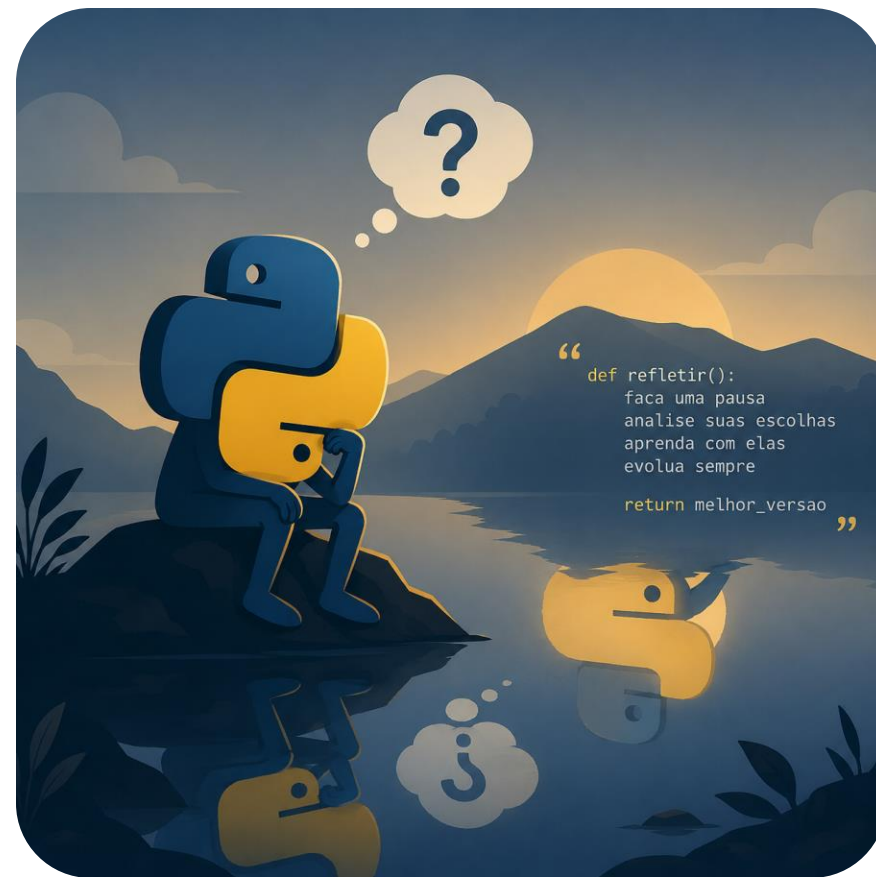


O que são Migrations?

O que é uma Migration?

Migration é uma instrução que diz ao Django:

"Crie ou altere esta tabela no banco de dados."

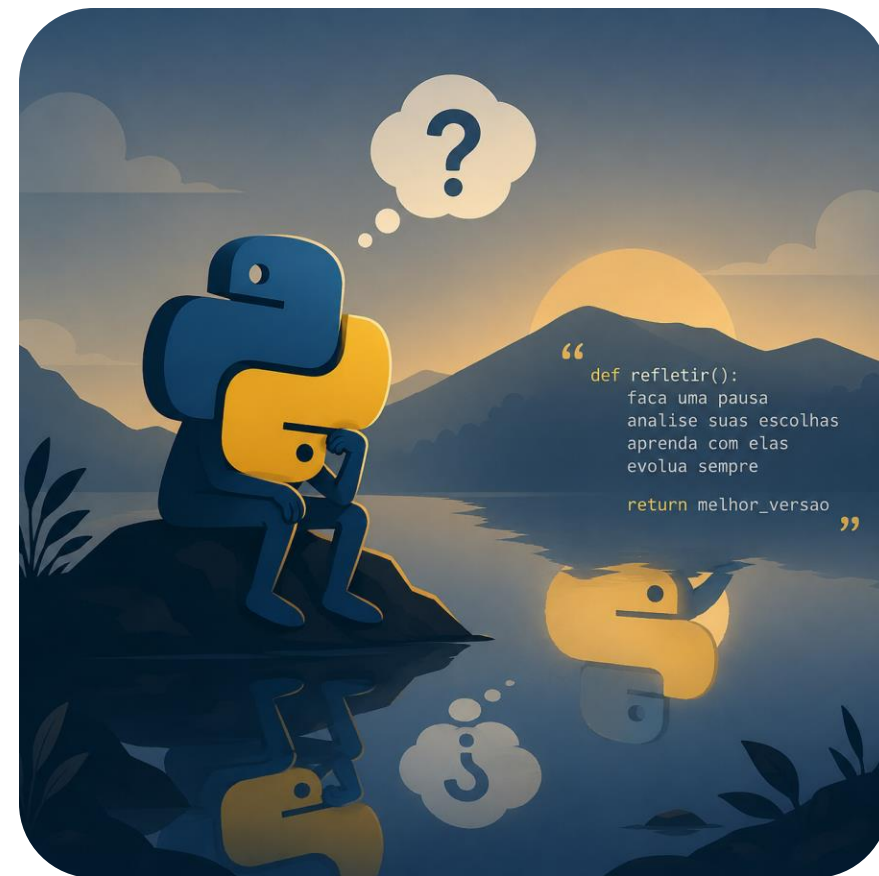


Vida Real

Imagine um engenheiro.

Primeiro ele faz a planta.

Depois os pedreiros constroem.



Comparação

Mundo Real

Planta

Construção

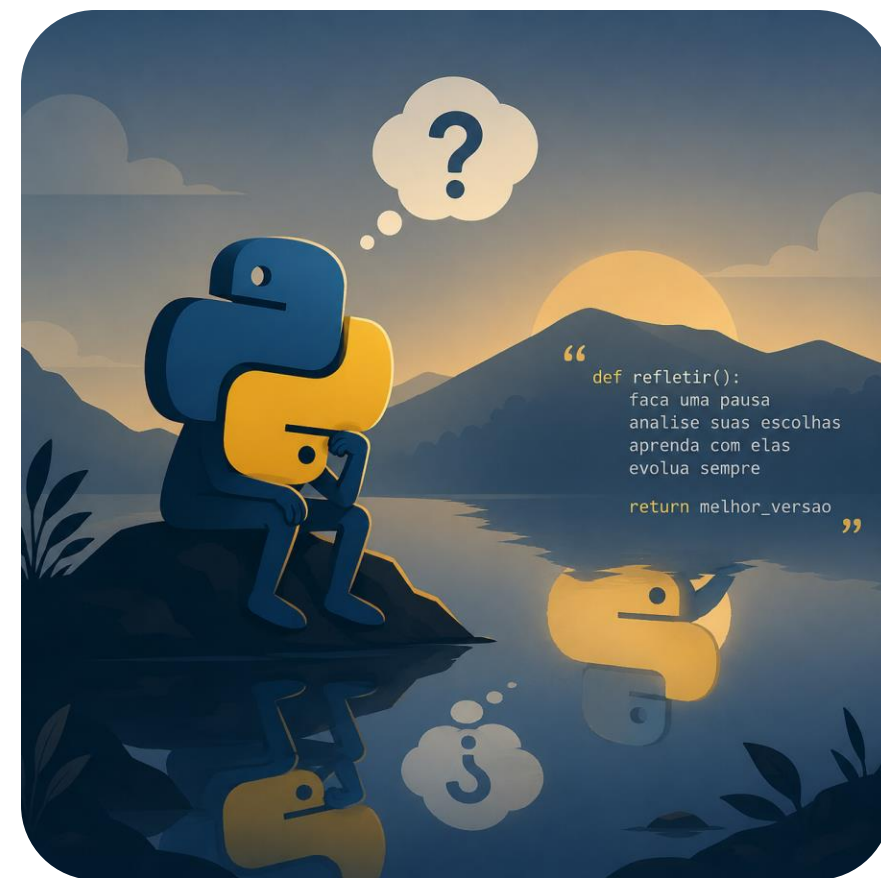
Casa pronta

Django

Model

Migration

Tabela

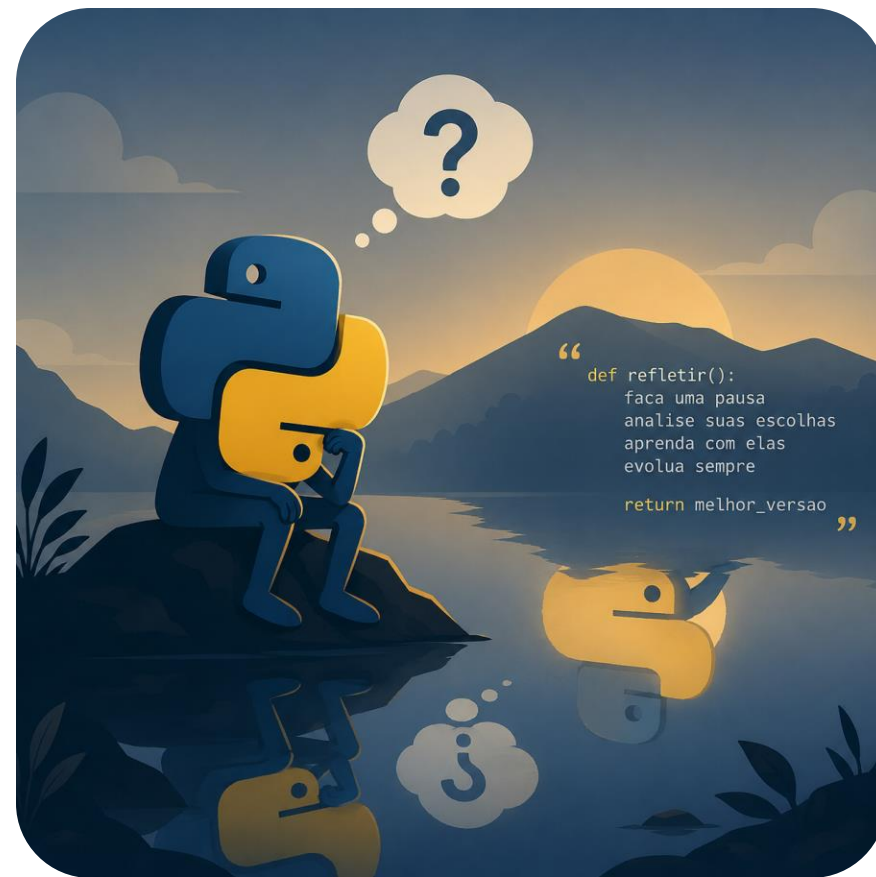


Criar Migration

Após criar o model:

`</>` Bash

```
python manage.py makemigrations
```



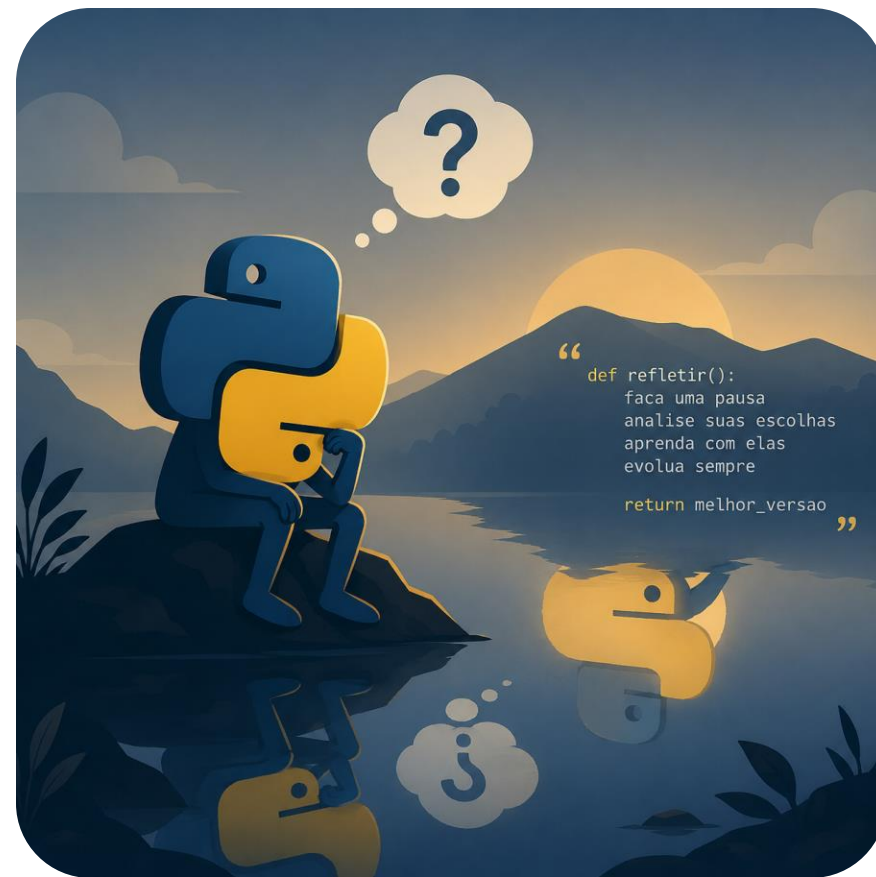
O que aconteceu?

O Django criou um arquivo dentro da pasta:

```
migrations/
```

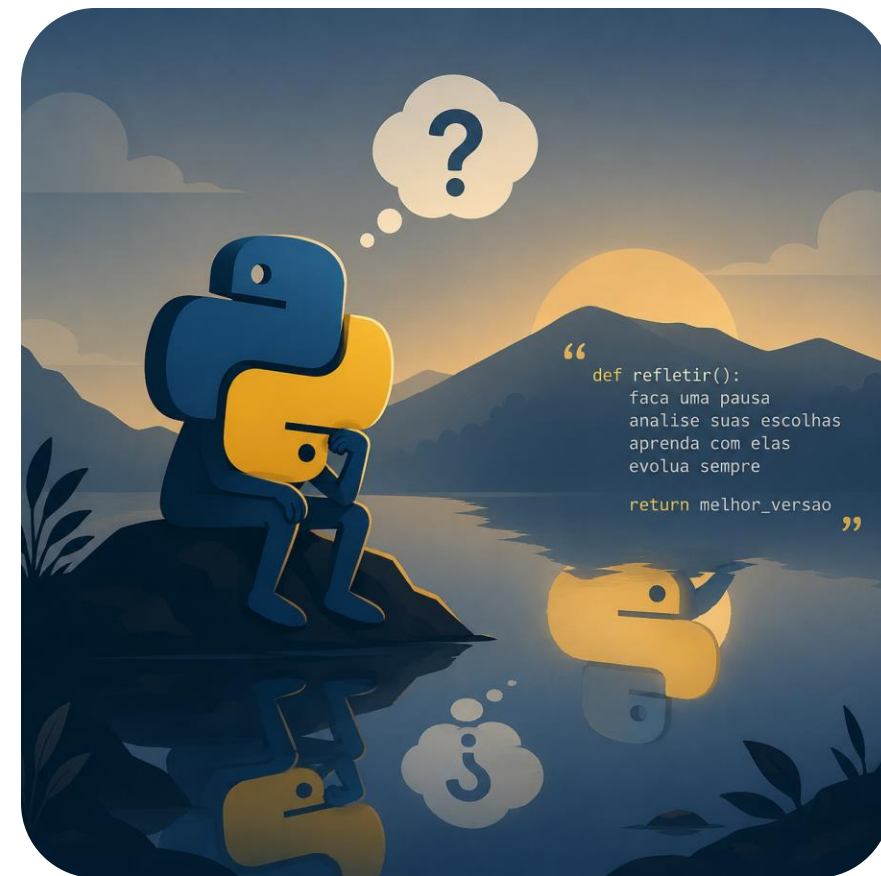
Exemplo

```
0001_initial.py
```



Reflexão

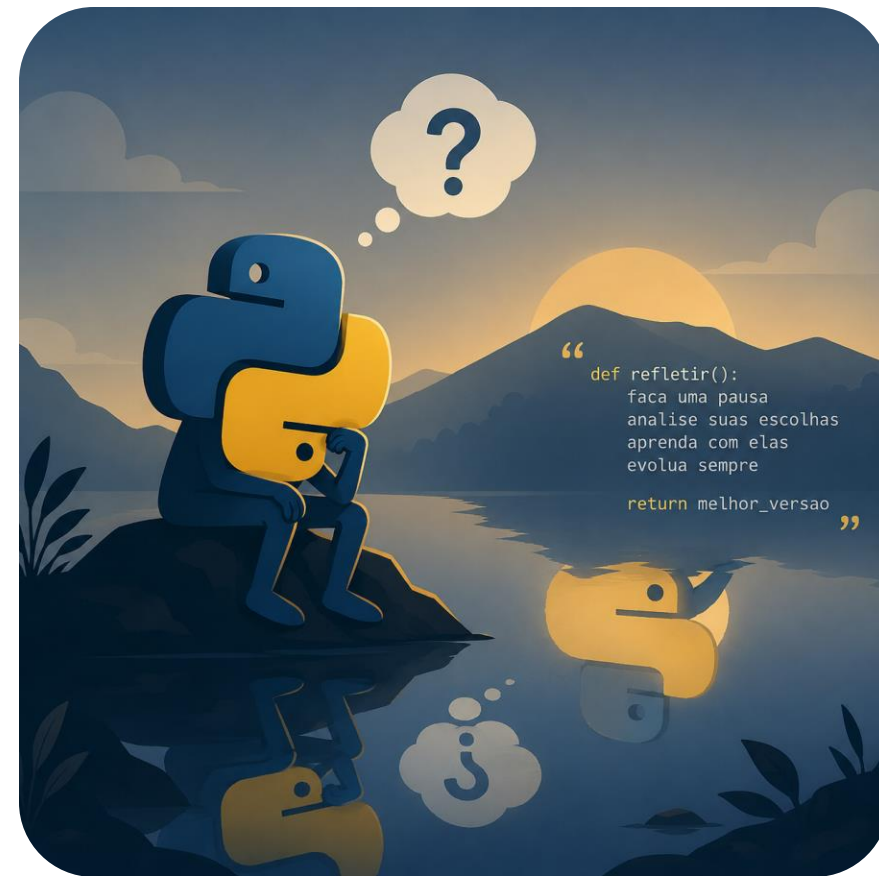
Esse arquivo contém as instruções para criar a tabela.



Aplicando no Banco

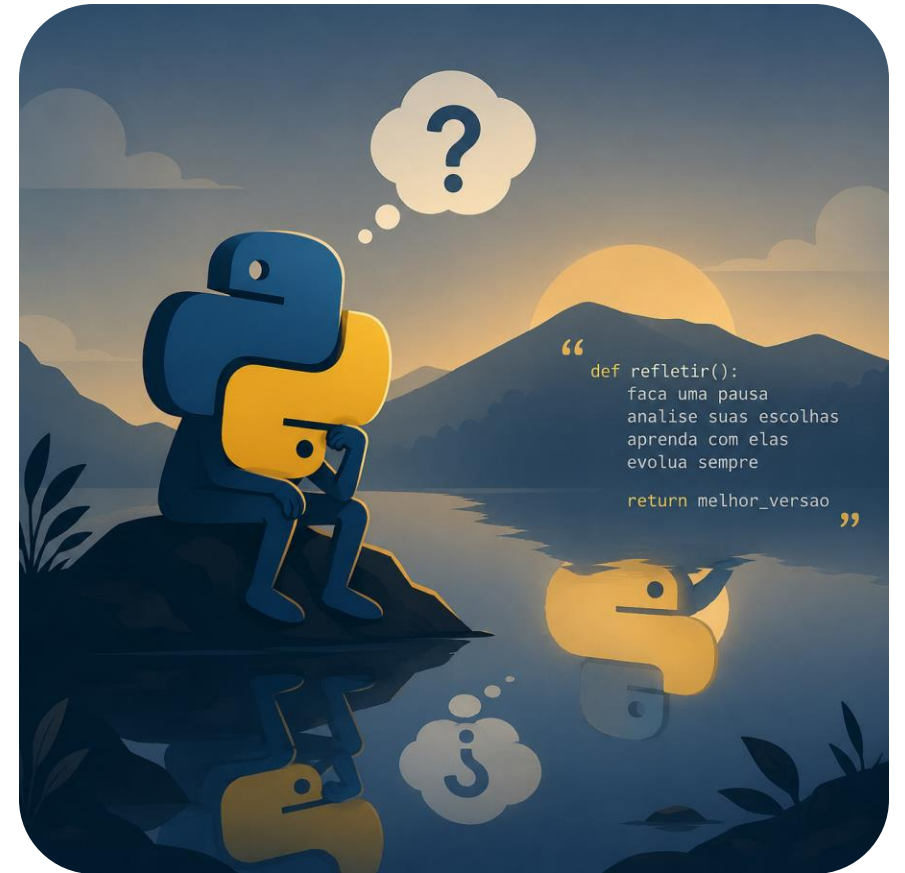
`</>` Bash

```
python manage.py migrate
```



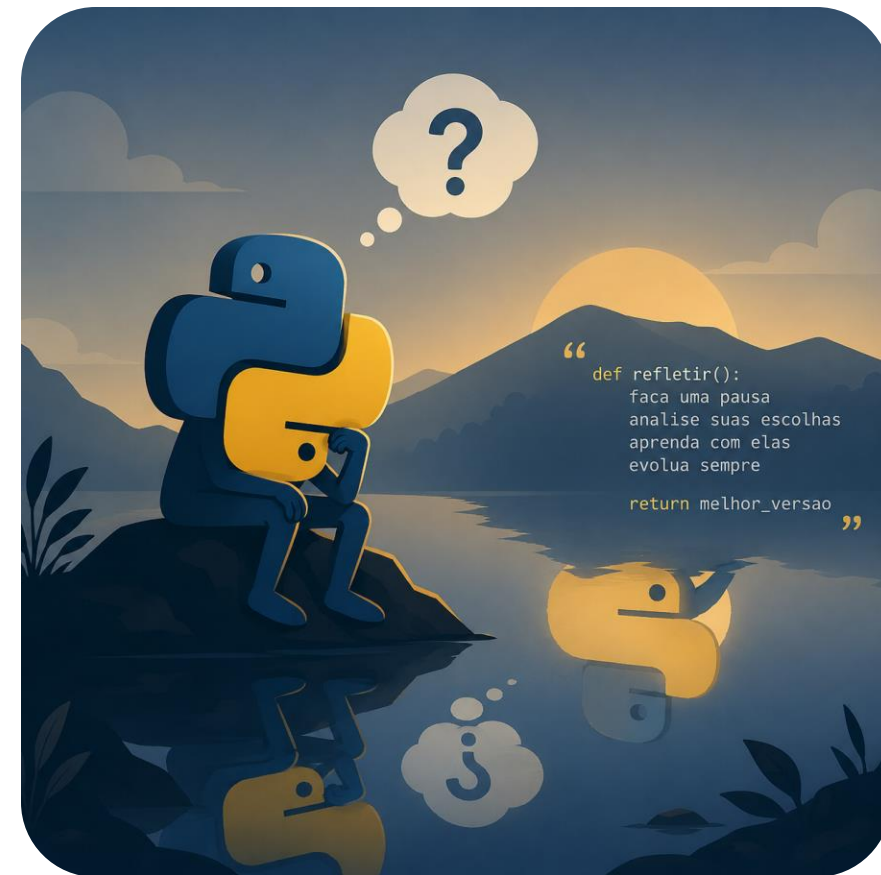
O que acontece?

O Django cria a tabela física no banco.



Fluxo Completo

```
Model
↓
makemigrations
↓
Migration
↓
migrate
↓
Banco de Dados
```





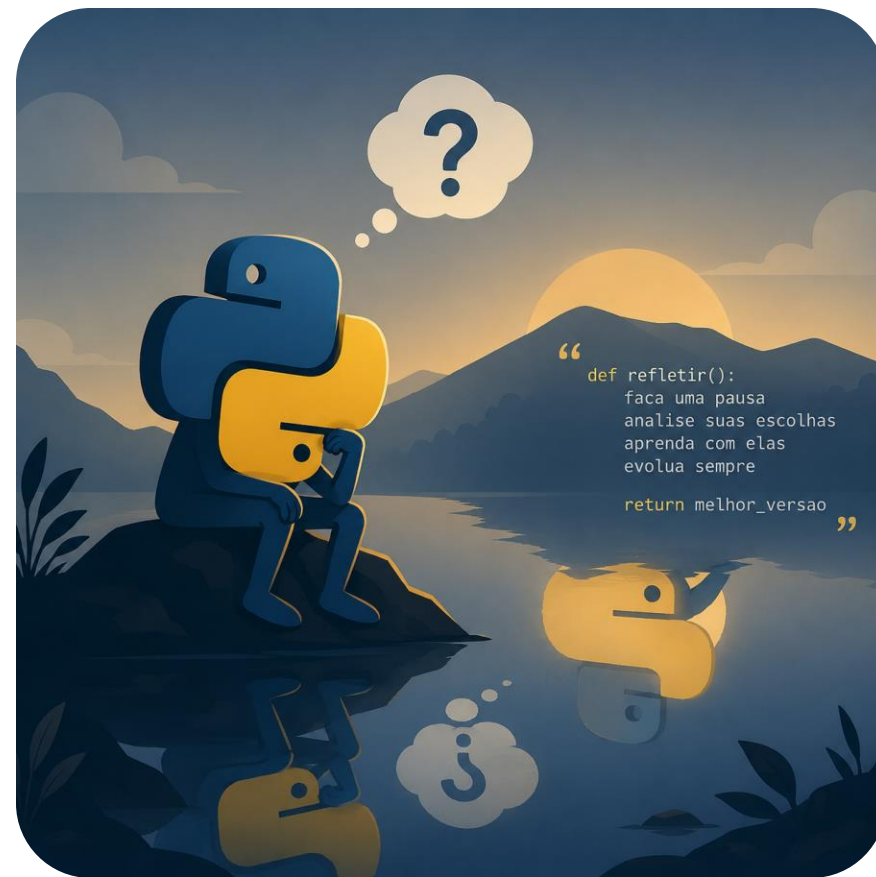
Relacionamentos no Django



Model Turma

</> Python

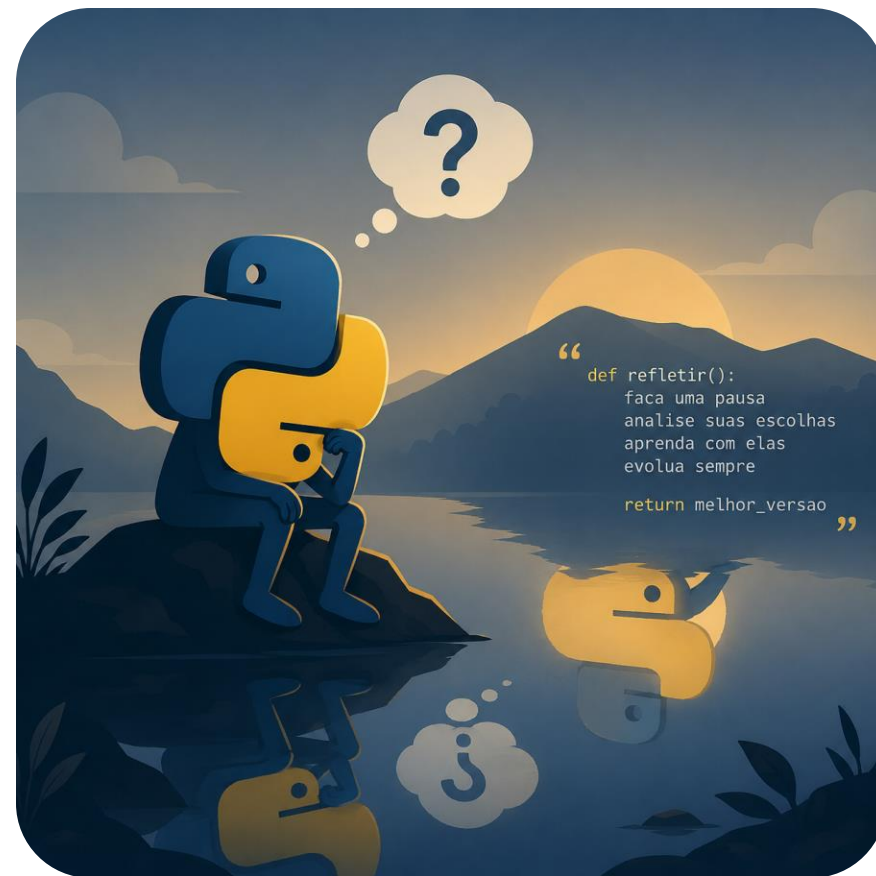
```
class Turma(models.Model):  
  
    nome = models.CharField(max_length=100)
```



Model Aluno

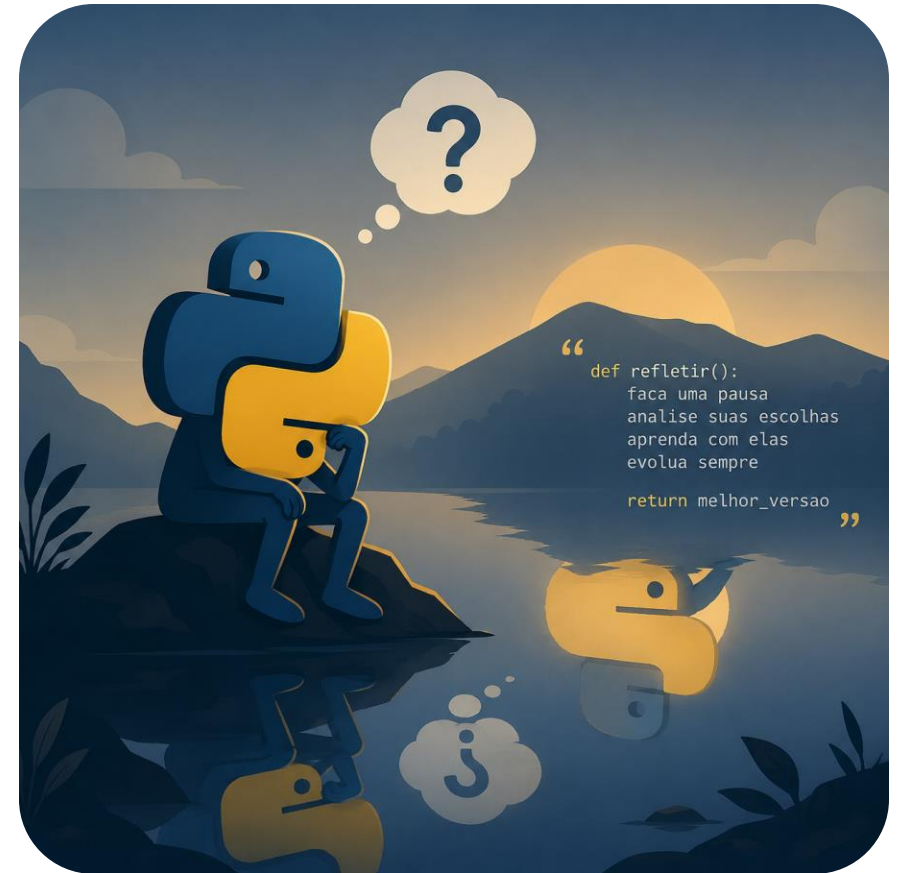
Python

```
class Aluno(models.Model):  
  
    nome = models.CharField(max_length=100)  
  
    turma = models.ForeignKey(  
        Turma,  
        on_delete=models.CASCADE  
    )
```



Reflexão

Um aluno pertence a uma turma.



```
“def refletir():  
    faça uma pausa  
    analise suas escolhas  
    aprenda com elas  
    evolua sempre  
    return melhor-versao”
```

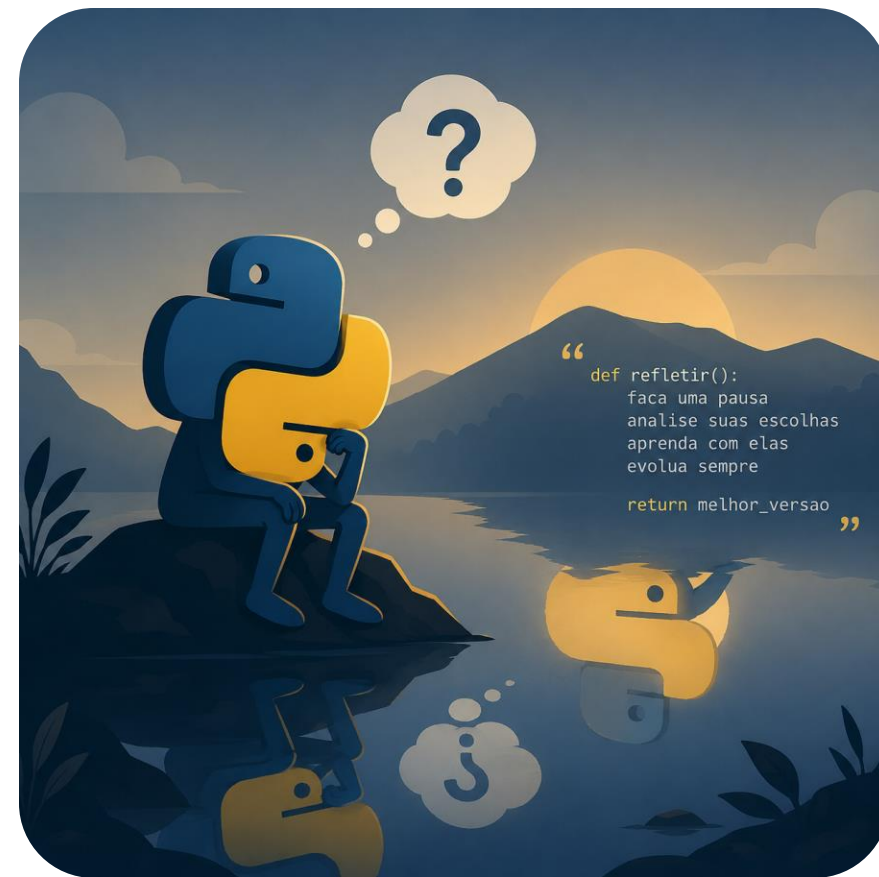

Exemplo 2 — Perfil e Usuário

`</>` Python

```
class Perfil(models.Model):  
  
    usuario = models.OneToOneField(  
        User,  
        on_delete=models.CASCADE  
    )
```

Reflexão

Um usuário possui apenas um perfil.



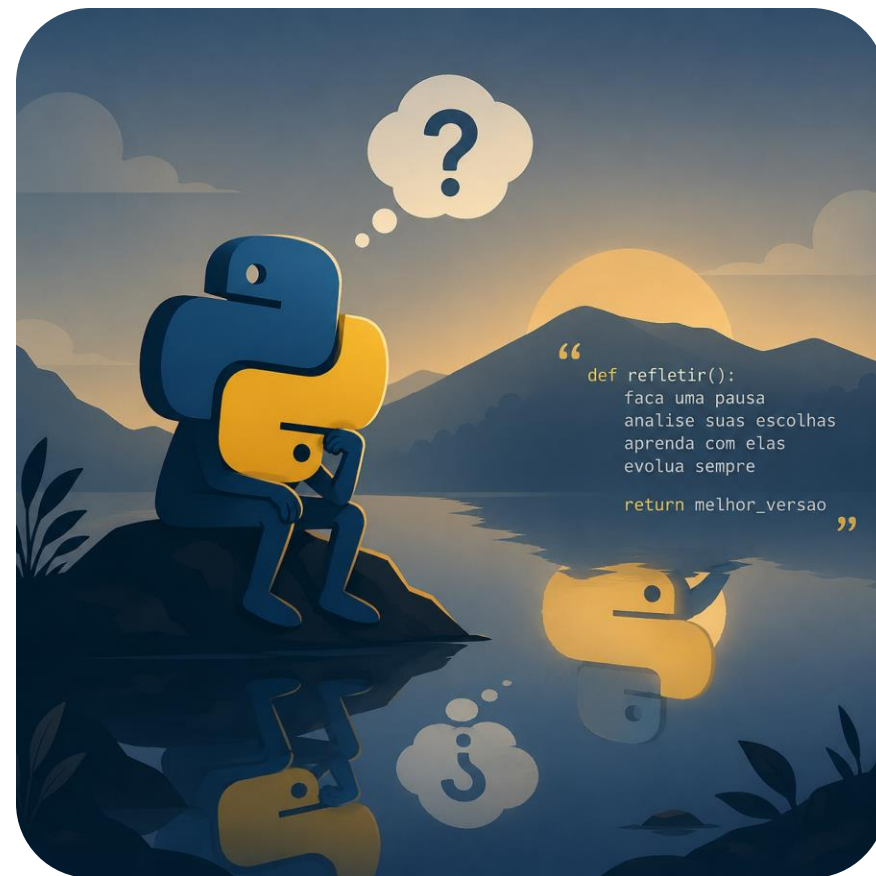
Exemplo 3 — Cursos e Alunos

</> Python

```
class Curso(models.Model):  
  
    nome = models.CharField(max_length=100)
```

</> Python

```
class Aluno(models.Model):  
  
    nome = models.CharField(max_length=100)  
  
    cursos = models.ManyToManyField(Curso)
```



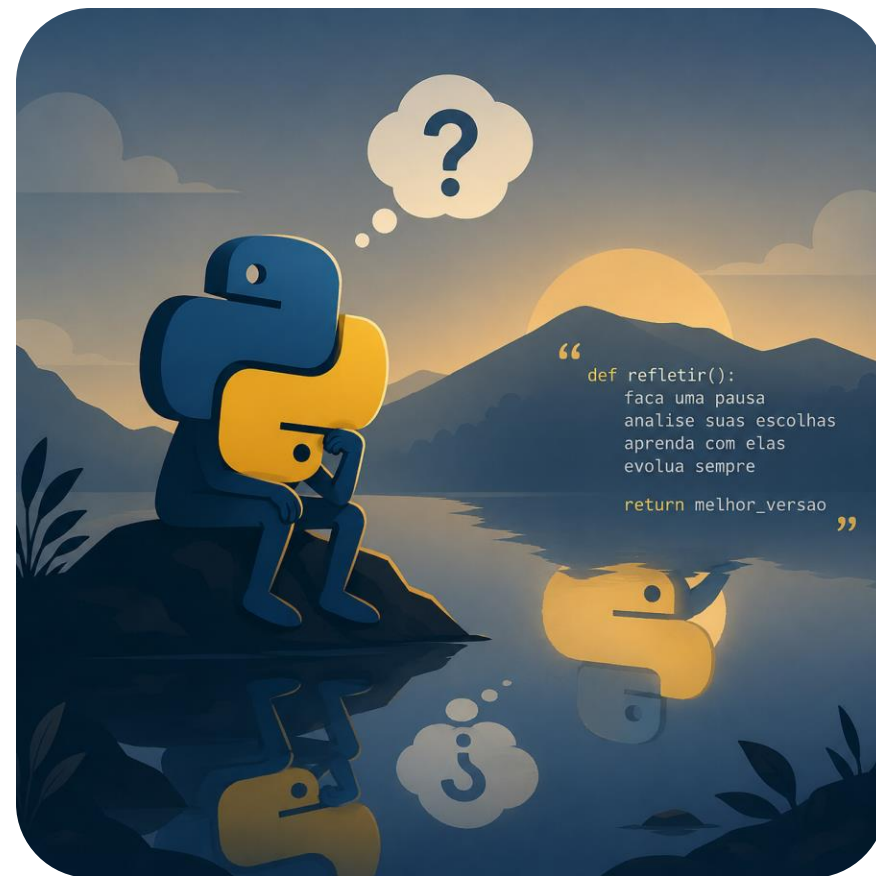
Exemplo 3 — Cursos e Alunos

</> Python

```
class Curso(models.Model):  
  
    nome = models.CharField(max_length=100)
```

</> Python

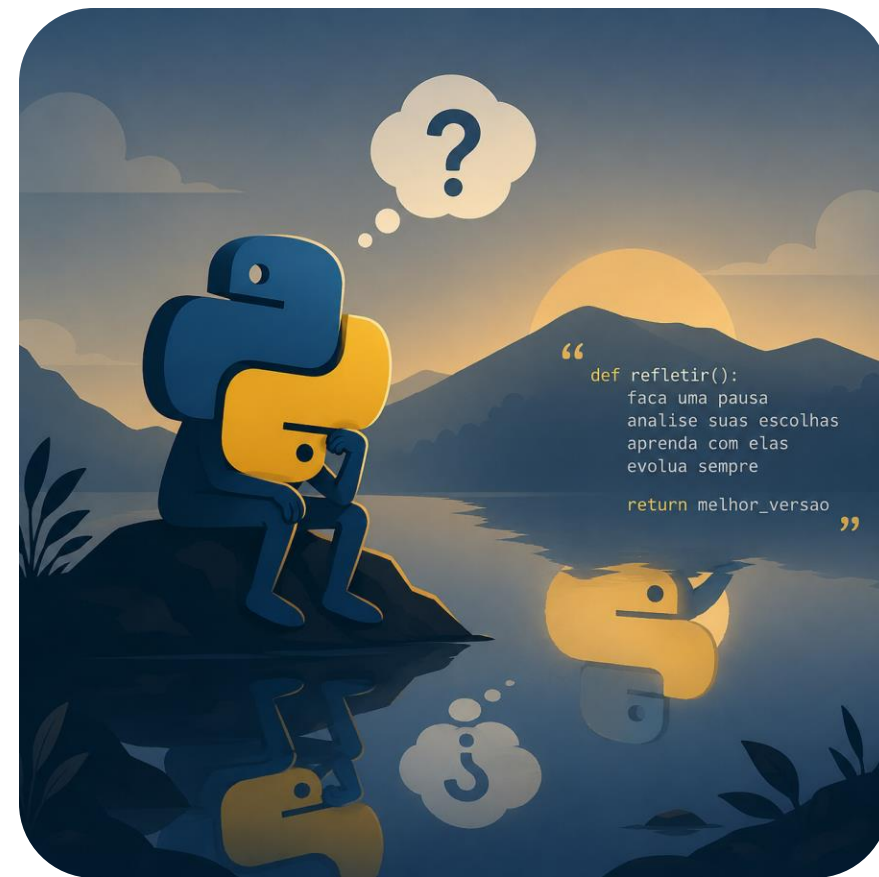
```
class Aluno(models.Model):  
  
    nome = models.CharField(max_length=100)  
  
    cursos = models.ManyToManyField(Curso)
```



Reflexão

Um aluno pode estudar vários cursos.

Um curso possui vários alunos.

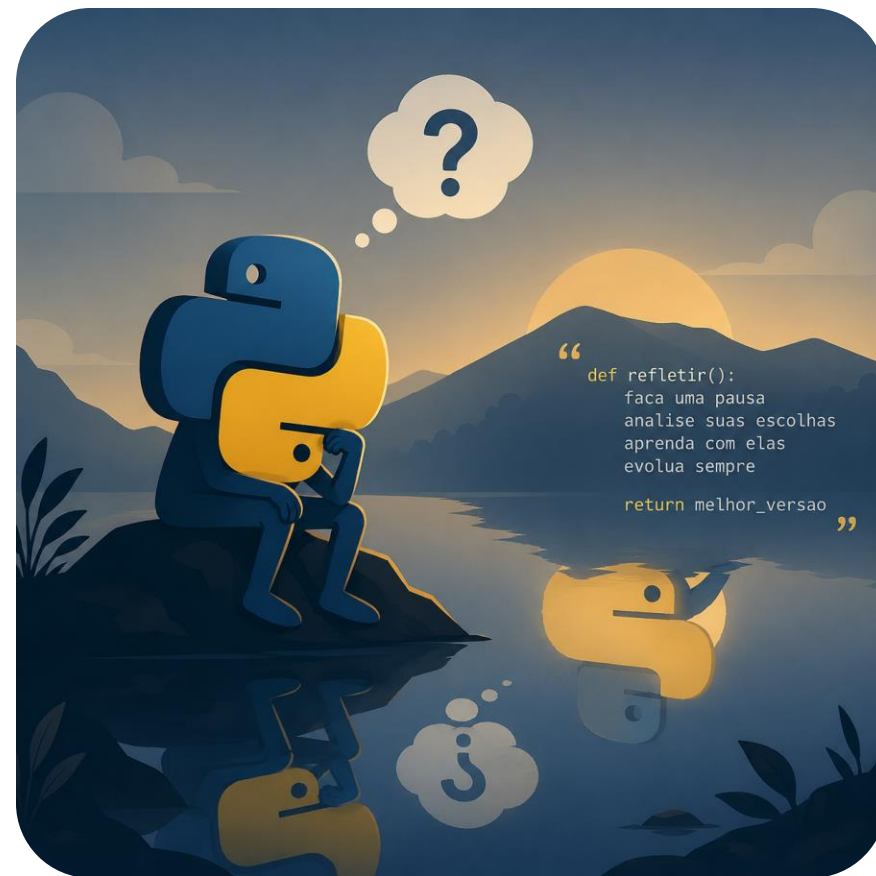


Exercício – Criação de Models, Migrations e Django Admin

Utilizando o **Sistema XPTO** desenvolvido nas aulas, crie uma nova funcionalidade contendo:

1. Pelo menos **um novo model**.
2. Gere e aplique as **migrations** para criar as tabelas no banco de dados.
3. Configure o **Django Admin** para gerenciar os registros desse model.
4. Cadastre pelo menos 5 registros para testar o funcionamento.

Entrega: Link do github no teams..



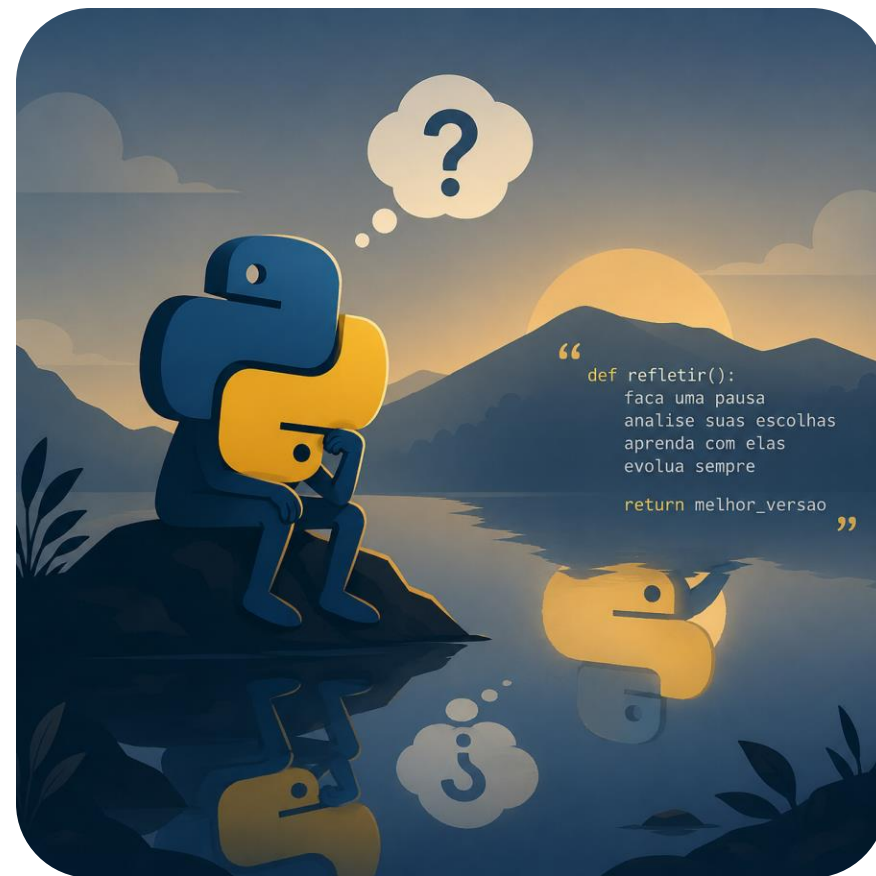
Exercício – Criação de Models, Migrations e Django Admin

Utilizando o **Sistema XPTO** desenvolvido nas aulas, crie uma nova funcionalidade contendo:

1. Desenvolva o model turma

2. Crie a relação de um para muitos com o model criado na aula passada.
3. Gere e aplique as **migrations** para criar as tabelas no banco de dados.
4. Configure o **Django Admin** para gerenciar os registros desse model.
5. Cadastre pelo menos 5 turmas
6. Gerar o template para exibir nome da turma, e nome do aluno.

Entrega: Link do github no teams..



ATÉ A
PRÓXIMA
AULA!